



Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Compiladores - Prova 2 (Max: 25 pts)
Professor: Pedro Ramos

Estudante: _____

1. **Gerador de Código I (9pts)** Suponha que você deve implementar um compilador para a seguinte linguagem simples, fortemente tipada, com tipos t , expressões e e programas p .

$t ::= \text{int}$	(tipo inteiro)
$t * t$	(tipo produto)
$e ::= n$	(inteiro literal)
$?$	(ler entrada inteira do usuário)
$e + e$	(adição)
$e - e$	(subtração)
(e, e)	(par)
$\text{fst } e$	(primeira projeção)
$\text{snd } e$	(segunda projeção)
$f(e)$	(aplicação de função)
$\text{let } x : t = e \text{ in } e \text{ end}$	(ligação de variável na expressão e)
$p ::= e$	
$\text{fun } f(x : t) : t = e; p$	(definição de função, recursão permitida)

Nos itens acima, x e f incluem identificadores. Por exemplo, aqui está um programa simples:

```
fun troca (p : int * int) : int * int = (snd p, fst p);  
fun trocaprod (p : int * (int * int)) : (int * int) * int =  
  (troca (snd p), fst p);  
trocaprod (?, (?, ?))
```

- (a) Descreva **detalhadamente** como seu compilador usará a pilha para implementar **chamadas de função e retornos**. Descreva quaisquer ponteiros auxiliares que você possa precisar. Há algo na linguagem acima que torna isso especialmente fácil? (4pts)
- (b) Descreva como você aloca espaço na pilha para um valor do tipo t . Qual ou quais instruções MIPS poderiam ser utilizadas? Dê exemplos. (1pt)
- (c) Descreva como seu compilador implementará expressões da forma (e_1, e_2) . Explique como a ordem de avaliação (da esquerda para a direita ou da direita para a esquerda) impacta suas escolhas. (2pts)
- (d) Descreva como seu compilador implementará expressões da forma $\text{fst } e$ e $\text{snd } e$. (2pts)

2. Gerador de Código II (9pts)

- (a) O que é uma pilha de tempo de execução e por que ela é importante para um desenvolvedor de compiladores? (1pt)
- (b) Para alguns compiladores, os registros de ativação (frames de pilha) contêm *links de controle* (ou "links estáticos"). Qual problema os *links de controle* são usados para resolver e como eles resolvem esse problema? (3pts)
- (c) Considere uma linguagem de programação de funções de primeira ordem que permite aninhamento de funções. Usando sintaxe semelhante a linguagem funcional, temos o seguinte código nesta linguagem:

```
let fun f(x) =  
  let  
    fun h(k) = k * x  
    fun g(z) = h(x + z + 1)  
  in  
    g(x + 1)  
  end  
in  
  f(17)  
end
```

Desenhe a pilha de chamadas desde a chamada de `f` até (e incluindo) a chamada da função `h`. Certifique-se de que todos os argumentos da função estejam incluídos no diagrama e indique claramente os *links de controle* se houver. (2pts)

- (d) Usando seu diagrama, explique como o código gerado a partir do corpo da função `h` pode acessar os valores associados às variáveis `k` e `x`. Em cada caso, deixe claro quais informações são conhecidas em tempo de compilação e quais informações são computadas em tempo de execução. (3pts)

3. **Tradução (4pts)** Explique como um compilador poderia traduzir literais inteiros positivos de 32bits (*unsigned int32*) para arquiteturas de 8, 16 e 64bits. Considere que o compilador pode ser implementado em qualquer uma das três arquiteturas. Considere também que o compilador deve ser capaz de gerar código para as 3 arquiteturas - ou seja, **deve haver regras semânticas para cada uma delas** (8bits, 16bits e 64bits).

Exemplo: imagine que o compilador deve traduzir uma expressão do tipo abaixo, que estouraria um *int32*:

```
1 let a = 4294967295
```

Quais regras semânticas devem ser implementadas para atender esse objetivo? Obs: é permitido o uso de condicionais e o uso de pseudo-código.

4. **Geração de Código Assembly (3pts)** Gere código MIPS para o programa abaixo. Comece do rótulo que delimita o início da função *f* (F_INICIO, por exemplo). Seu programa deve manter a **semântica** do programa original.

```
1 int f() {
2     int i = 0;
3     while (i < 10) {
4         int j = i + 1
5         for (; j < i - 1; j++) {
6             if j == 0 {
7                 return -10
8             }
9         }
10        i++
11    }
12
13    return i
14 }
```